



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра прикладной математики

КУРСОВАЯ РАБОТА

по дисциплине

Языки программирования для статистической обработки данных

Тема курсовой работы: Реализация алгоритма случайного леса.
Случайный лес в задачах регрессионного анализа.

Студентка группы ИМБО-01-20

Черных Анастасия Николаевна

(подпись студента)

Руководитель курсовой работы

Митина Ольга Алексеевна

(подпись руководителя)

Работа представлена к защите

«__» _____ 2022 г.

Допущен к защите

« » _____ 2022 г.

Москва 2022



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра прикладной математики

ЗАДАНИЕ

на выполнение курсовой работы по дисциплине

Языки программирования для статистической обработки данных

Студентка Черных Анастасия Николаевна Группа ИМБО-01-20

Тема работы: Реализация алгоритма случайного леса. Случайный лес в задачах регрессионного анализа.

Исходные данные: RStudio, техническая документация, методическое пособие.

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Реализация алгоритма случайного леса
2. Применение алгоритма случайного леса в задачах регрессии

Срок представления к защите до «___» _____ 2022 г.
курсовой работы:

Задание на курсовую _____ (Митина О.А.)
работу выдал
«___» _____ 2022 г.

Задание на курсовую _____ (Черных А.Н.)
работу получил

Москва 2022

ОТЗЫВ

на курсовую работу по дисциплине

Языки программирования статистической обработки данных

Студент Черных Анастасия Николаевна группы ИМБО-01-20

(ФИО студента)

Характеристика курсовой работы

Критерий	Да	Нет	Не полностью
1. Соответствие содержания курсовой работы указанной теме			
2. Соответствие курсовой работы заданию			
3. Соответствие рекомендациям по оформлению текста, таблиц, рисунков и пр.			
4. Полнота выполнения всех пунктов задания			
5. Логичность и системность содержания курсовой работы			
6. Отсутствие фактических грубых ошибок			

Замечания:

Рекомендуемая оценка:

_____ Митина О.А.,

подпись руководителя

ФИО руководителя

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	4
ВВЕДЕНИЕ	5
1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
1.1 Регрессионный анализ	6
1.2 Алгоритм «Случайный лес»	10
2. ПРАКТИЧЕСКАЯ ЧАСТЬ	13
2.1 Регрессионный анализ с помощью «Случайного леса»	13
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	20
ПРАКТИЧЕСКАЯ ЧАСТЬ	21
ПРИЛОЖЕНИЯ	22
Приложение А	23

ВВЕДЕНИЕ

Случайный лес является мощным алгоритмом машинного обучения, который использует комбинацию решающих деревьев для решения задач классификации и регрессии. Он является одним из наиболее популярных методов ансамблевого обучения и широко применяется в различных областях, включая биоинформатику, финансовую аналитику, медицинскую диагностику и другие.

Основная идея случайного леса заключается в создании ансамбля решающих деревьев, где каждое дерево строится на основе случайной подвыборки обучающих данных и случайного подмножества признаков. Это делается для уменьшения корреляции между деревьями и увеличения разнообразия моделей в ансамбле.

Данная работа представляет практическое применение случайного леса в линейной регрессии. А именно, для классификации музыкальных жанров, а также демонстрации его эффективности в решении подобных задач.

Цель данной курсовой работы — реализация алгоритма «Случайный лес» и изучение его применения в задачах линейной регрессии

Задачи, решаемые в ходе курсовой работы:

- изучена документация и техническая литература по исследуемой теме;
- реализован алгоритм регрессионного анализа при использовании метода «Случайного леса»
- разработан программный код приложения с использованием языка программирования R;
- приложение протестировано и результаты собраны;
- изучены правила качественного оформления документации.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Регрессионный анализ

Регрессия (лат. regressio — обратное движение) в статистике — статистическая зависимость среднего значения случайной величины от значений другой случайной величины или нескольких случайных величин; введена Фрэнсисом Гальтоном в 1886 году.[1]

Как и корреляционный анализ, регрессионный анализ отражает только количественные зависимости между переменными. Причинно-следственные зависимости регрессионный анализ не отражает. Гипотезы о причинно-следственной связи переменных должны формулироваться и обосновываться исходя из теоретического анализа содержания изучаемого явления.

Цель регрессионного анализа — с помощью уравнения регрессии предсказать ожидаемое среднее значение результирующей переменной.

Основные задачи регрессионного анализа следующие:

- определения вида и формы зависимости;
- оценка параметров уравнения регрессии;
- проверка значимости уравнения регрессии;
- проверка значимости отдельных коэффициентов уравнения;
- построение интервальных оценок коэффициентов;
- исследование характеристик точности модели;
- построение точечных и интервальных прогнозов результирующей переменной.

Рассмотрим основные задачи регрессионного анализа:

1. Установление формы зависимости.
2. Определение функции регрессии.
3. Оценка неизвестных значений зависимой переменной.

4. Уравнение регрессии.

Область построения регрессионной модели — это, в каком-то смысле, кульминация с точки зрения математики, потому что в ней сливаются сразу все возможные разделы, которые могут быть полезны в анализе данных, например:

- линейная алгебра со способами представления данных;
- математический анализ с теорией оптимизации и средствами анализа функций;
- теория вероятности со средствами описания случайных событий и величин и моделирования зависимости между переменными.

В отличие от функциональной зависимости $y = f(x)$, которая каждому значению независимой переменной x ставит в соответствие одно определённое значение величины y , при регрессионной зависимости одному и тому же значению x могут соответствовать различные значения величины y . Если при каждом значении $x = x_i$ наблюдается n_i значений $y_{i_1}, \dots, y_{i_{n_i}}$ величины y , то зависимость среднего арифметического (1.1)

$$\langle y \rangle = \frac{y_{i_1} + \dots + y_{i_{n_i}}}{n_i}, \quad (1.1)$$

от $x_i, i = 1, \dots, N$ и является средней регрессией. Различные виды линейных регрессий представлены на Рисунке 1.1. Заметим, что это лишь частные случаи. На деле, линейные зависимости могут иметь и более сложные формы.

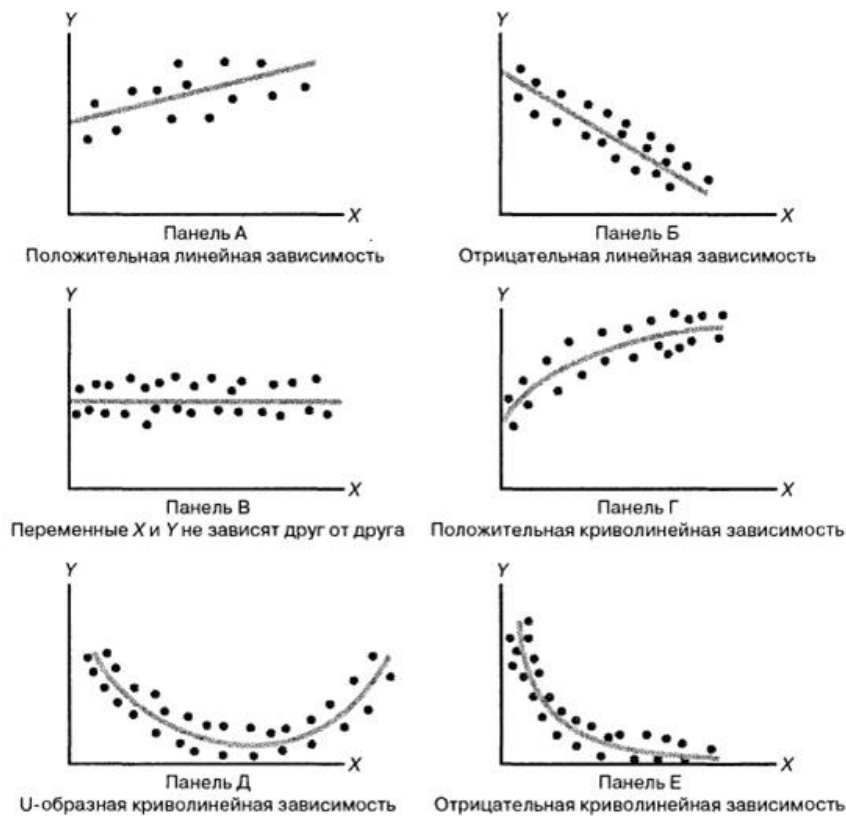


Рисунок 1.1 — Примеры линейных зависимостей

Самое непростое при формировании регрессионной модели — изначально заложить в её память какую-то конкретную функцию. Выбрав конкретную зависимость, мы сможем использовать регрессионные методы для калибровки модели.[2]

Если остановиться на линейных регрессионных моделях, то мы предполагаем, что есть некий набор реальных значений (см. Рисунок 1.2)

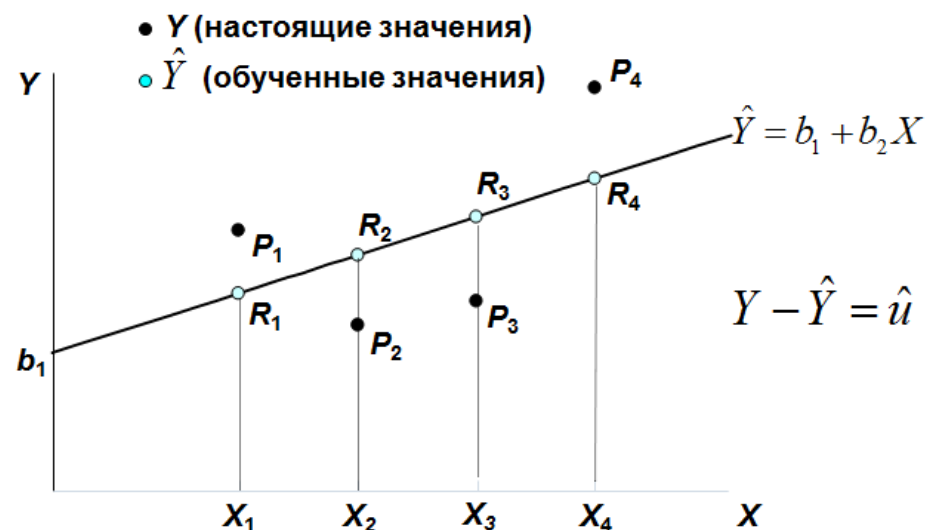


Рисунок 1.2 — График наблюдаемых и полученных значений

На рисунке показаны наблюдаемые значения X_1, X_2, X_3, X_4 . Для каждого из «иксов» известно значение Y (в нашем случае, это точки: P_1, P_2, P_3, P_4). Это точки, которые мы наблюдаем из данных. И, в ходе регрессионного анализа, мы решили, что линейная регрессия лучше всего описывает зависимость между X и Y .

Дальше весь вопрос заключается в том, как построить уравнение прямой линии (1.2)

$$Y = b_1 + b_2 X, \quad (1.2)$$

где b_2 — коэффициент наклона, b_1 — коэффициент пересечения.

Весь вопрос заключается в том, какие b_2 и b_1 лучше всего установить, чтобы эта прямая линия как можно точнее описывала зависимость между этими переменными.[3]

Точки R_1, R_2, R_3, R_4 — это значения, которые выдаёт наша модель при значениях X . Модель будет считаться качественной тогда и только тогда, когда точки R максимально близки к точкам P .

Если мы построим расстояние между этими точками для одинаковых x , то мы получим то, что называется ошибками линейной регрессии. Мы получим отклонения в линейной регрессии, и эти отклонения называются $U_1, U_2, U_3 \dots U_n$. Эти ошибки могут быть как положительными, так и отрицательными. Чтобы эти отклонения сравнить, их нужно проанализировать — для этого возведём их в квадрат.

Сумма квадратов всех отклонений в математической статистике называют RSS (Residual Sum of Squares). Минимизировав RSS по b_1 и минимизировав RSS по b_2 , мы получаем оптимальные коэффициенты, которые фактически выводятся методом наименьших квадратов.[4]

Однако, выбор функции становится основной проблемой в задачах линейной регрессии. Тут нам на помощь приходит метод «случайного леса».

1.2 Алгоритм «Случайный лес»

Случайный лес — один из самых потрясающих алгоритмов машинного обучения, придуманный Лео Брейманом и Адель Катлер ещё в прошлом веке. Он дошёл до нас в первозданном виде, так как никакие эвристики не смогли его существенно улучшить и является одним из немногих универсальных алгоритмов. Универсальность заключается, во-первых, в том, что он хорош во многих задачах, во-вторых, в том, что есть случайные леса для решения задач классификации, регрессии, кластеризации, поиска аномалий, селекции признаков и т.д.

Алгоритмы случайного леса имеют три основных гиперпараметра, которые необходимо задать перед обучением. К ним относятся размер узла, количество деревьев и количество выбранных объектов. После этого классификатор случайного леса можно использовать для решения задач регрессии или классификации.[5]

Процесс построения случайного леса из деревьев включает следующие шаги:

1. Случайная выборка: Из обучающего набора данных случайным образом выбирается подмножество данных с возвращением (bootstrap sample). Это означает, что некоторые экземпляры данных могут быть выбраны несколько раз, а некоторые могут быть пропущены.
2. Случайные признаки: Из общего набора признаков случайным образом выбирается подмножество признаков. Это делается для каждого дерева в ансамбле. Выбор случайных признаков помогает уменьшить корреляцию между деревьями и увеличить разнообразие моделей.
3. Построение деревьев: Для каждого дерева в случайном лесу выполняется процесс построения решающего дерева. На каждом узле дерева происходит разделение данных на основе одного из случайно

выбранных признаков. Разделение основано на некотором критерии, таком как индекс Джини или энтропия, и целью является максимизация прироста информации или уменьшение неопределенности.

4. Прогнозирование: После построения всех деревьев в случайном лесу, прогноз для новых данных выполняется путем агрегирования прогнозов каждого дерева (см. Рисунок 1.3). В задачах классификации, прогноз может быть определен большинством голосов классов, а в задачах регрессии- средним или медианным значением прогнозов.

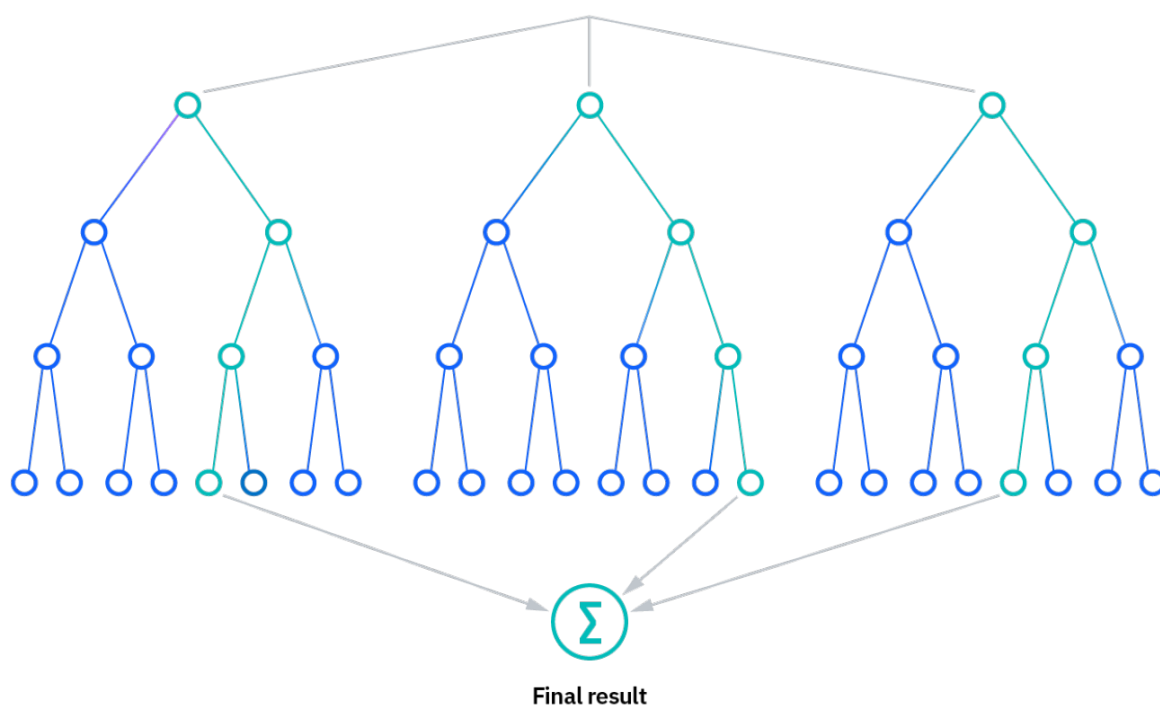


Рисунок 1.3 — Общий вид случайного леса

Понятно, что такая схема построения соответствует главному принципу ансамблирования: базовые алгоритмы должны быть хорошими и разнообразными. Поэтому каждое дерево строится на своей обучающей выборке и при выборе расщеплений есть элемент случайности.[6]

Преимущества случайного леса включают его способность обрабатывать большие объемы данных, устойчивость к выбросам и шуму, а также способность автоматически обрабатывать отсутствующие значения. Он также

обладает хорошей обобщающей способностью и может быть эффективно использован для классификации и регрессии.[7]

Однако, случайный лес может быть склонен к переобучению, особенно если количество деревьев в ансамбле слишком велико. Также, поскольку каждое дерево строится на подмножестве данных и признаков, интерпретация результатов может быть сложной.

Отметим ряд ключевых преимуществ и проблем, которые представляет собой алгоритм случайного леса при использовании для задач классификации или регрессии.[8]

Ключевые преимущества:

- снижение риска переобучения;
- обеспечивает гибкость;
- легко определить важность функции.

Ключевые проблемы:

- трудоемкий процесс;
- требуется больше ресурсов[9];
- более сложный.

В целом, случайный лес является мощным алгоритмом машинного обучения, который может быть эффективно применен в различных областях, включая биоинформатику, финансовую аналитику, медицинскую диагностику и другие.[10]

Применим представленную теоретическую базу на реальном примере данных.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Регрессионный анализ с помощью «Случайного леса»

Для реализации практической части [1] данной работы, найдём подходящие данные для анализа. Моё внимание привлекла база данных песен с известного стримингового сервиса Spotify. Каждая песня имеет такие параметры, как:

- `artist_name` — имя исполнителя;
- `track_name` — название песни;
- `popularity` — популярность;
- `acousticness` — акустичность;
- `danceability` — танцевальность;
- `duration_ms` — длительность (в миллисекундах);
- `energy` — энергия;
- `instrumentalness` — инструментальность;
- `key` — ключ;
- `liveness` — живость;
- `mode` — лад;
- `speechiness` — красноречивость;
- `tempo` — темп;
- `valence` — валентность;
- `music_genre` — музыкальный жанр.

Предварительно скачав датасет с открытого информационного ресурса Kaggle[2], загрузим набор данных в RStudio из файла "music_genre.csv" с помощью функции `read.csv()`. Загруженные данные сохраним в переменной `dataset`.

Затем произведём необходимые преобразования, чтобы упростить чтение набора данных, а именно, из набора данных удалим строки, где хотя бы одно значение равно 0, с помощью функции `subset()`. Это делается для удаления неполных или некорректных данных. Далее, из набора данных удаляются столбцы 1 и 16 с помощью операции `[, -c(1, 16)]`. Это делается, чтобы удалить ненужные или неинформативные признаки. Столбец "music_genre" преобразуем в факторный тип данных с помощью функции `as.factor()`. Также используем функция `table()` для подсчета количества записей для каждого значения в столбце "music_genre". Применим функцию `sapply()` к набору данных, чтобы увидеть типы данных для каждого столбца. Исходя из вывода, принимаем решение о преобразовании в факторный тип данных столбцов «key» и «mode»[3].

Воспользуемся функцией `str()` для вывода структуры набора данных, чтобы увидеть типы и размеры переменных. В выводе видим таблицу, формат которой подходит для дальнейшего анализа (см. Рисунок 2.1)

	artist_name	track_name	popularity	acousticness	danceability	duration_ms	energy	instrumentalness	key	liveness	loudness
1	Thievery Corporati...	The Shining Path	31	1.27e-02	0.622	218293	0.890	9.50e-01	D	0.1240	
2	Dillon Francis	Hurricane	28	3.06e-03	0.620	215613	0.755	1.18e-02	G#	0.5340	
3	Dubloadz	Nitro	34	2.54e-02	0.774	166875	0.700	2.53e-03	C#	0.1570	
4	What So Not	Divide & Conquer	32	4.65e-03	0.638	222369	0.587	9.09e-01	F#	0.1570	
5	Jordan Comolli	Clash	46	2.89e-02	0.572	214408	0.803	7.74e-06	B	0.1060	
6	Hraach	Delirio	43	2.97e-02	0.809	416132	0.706	9.03e-01	G	0.0635	
7	Kayzo	NEVER ALONE	39	2.99e-03	0.509	292800	0.921	2.76e-04	F	0.1780	
8	Shlump	Lazer Beam	22	9.34e-03	0.578	204800	0.731	1.12e-02	A	0.1110	
9	Chase & Status	Lost & Not Found - ...	30	8.55e-01	0.607	170463	0.158	0.00e+00	F#	0.1060	
10	G Jones	Mind	27	3.37e-02	0.513	165132	0.828	5.69e-01	B	0.1090	
11	Champagne Drip	Satellite (Feat. Len X)	31	1.60e-02	0.660	236089	0.892	6.05e-04	C	0.1630	
12	Getter	Bonesaw	27	1.94e-04	0.601	243857	0.969	7.76e-01	D	0.4090	
13	Excision	Bring the Madness ...	50	3.31e-03	0.391	240000	0.957	2.20e-01	G	0.1120	
14	Dillon Francis	Without You - The ...	29	1.58e-01	0.660	266282	0.784	2.79e-04	C	0.0978	
15	The Bloody Beetro...	Irreversible (feat. A...	35	6.02e-05	0.259	185333	0.687	8.62e-03	G#	0.3380	
16	empty_field	7th Sevens	50	2.81e-02	0.656	307328	0.653	8.58e-01	F#	0.1120	

Рисунок 2.1 — Преобразованный для анализа массив данных

Получив нужный формат данных, разобьём их на обучающую и тестовую выборку. Для этого, с помощью функции `set.seed()` установим начальное значение для генератора случайных чисел, чтобы результаты были воспроизводимыми. Воспользуемся функцией `sample()` — она разделит данные так, что обучающая выборка составит около 70% от общего количества

записей, а тестовая выборка — около 30%. Количество данных, попавших в каждую из выборок, представлено на Рисунке 2.2

```
> nrow(train)
[1] 28453
> nrow(test)
[1] 12107
```

Рисунок 2.2 — Разбиение данных на обучающую и тестовую выборку

Загружаем в RStudio пакет `randomForest` при помощи команды `library(randomForest)`. С помощью функции `randomForest()` построим модель случайного леса. Отметим, что формула указывает, что столбец «`music_genre`» является зависимой переменной, а все остальные столбцы являются независимыми переменными. Таким образом, мы проанализируем, как засчёт комбинации различных параметров можно предугадать музыкальный жанр песни. Отметим, что для построения модели используется обучающая выборка.

Проанализируем зависимость параметров, на построенную модель (см. Рисунок 2.3)

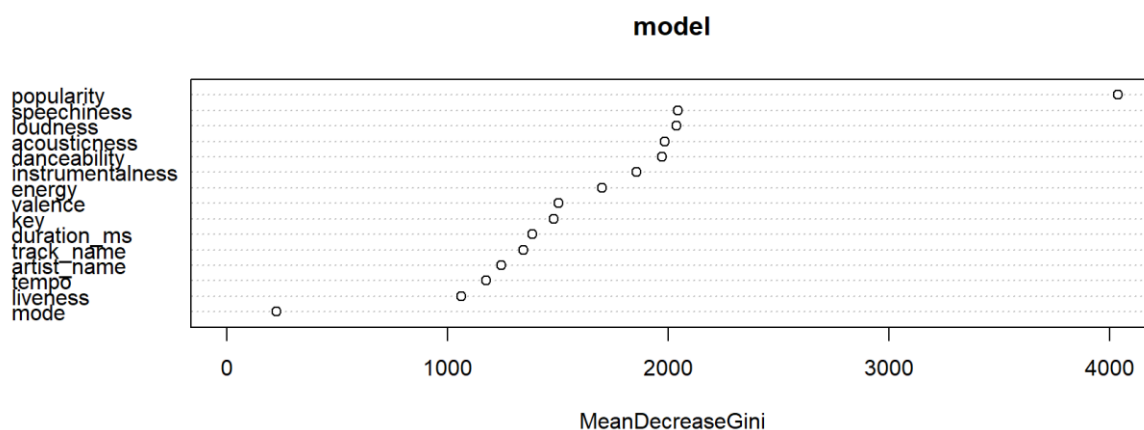


Рисунок 2.3 — Влияние параметров на определение параметра «`music_genre`»

Отметим, что наибольшее влияние оказывает параметр «`popularity`», что объясняется тем, что многие жанры действительно выделяются именно засчёт своей массовости. Тогда как значение параметра «`mode`» сильно уступает другим. Действительно, люди склонны испытывать разные эмоции, поэтому в равной степени в любом жанре есть и мажорные, и минорные композиции.

Чтобы проверить свою гипотезу, воспользуемся функцией `tuneRF()` для настройки параметров модели случайного леса. Функция произведёт подбор оптимального количества деревьев, количества признаков для каждого разделения, шага изменения количества признаков, улучшения и трассировки.

Спрогнозируем новые значения с использованием настроенной модели. Результаты прогноза сохраним в переменной `result_tuned` в виде фрейма данных. Вновь проверим точность прогноза по описанной выше схеме. В данном случае, получаем на выходе параметр `comprate`, равный 0.8559315, что $\approx$ 86%. Данный результат $>$ 85%, поэтому мы можем считать его удовлетворительным.

Следовательно, модель, построенная в ходе данной работы, с высокой точностью может предсказывать музыкальный жанр композиции по заданным параметрам.

ЗАКЛЮЧЕНИЕ

В заключение, случайный лес является мощным и эффективным алгоритмом машинного обучения, который позволяет решать задачи классификации и регрессии. Его способность работать с большими объемами данных, а также устойчивость к шуму и выбросам делают его привлекательным инструментом для решения различных задач.

Случайный лес обладает высокой обобщающей способностью, что позволяет ему хорошо работать на новых данных. Он также способен автоматически выявлять важные признаки, что делает его полезным для анализа данных.

Стоит отметить, что при использовании случайного леса необходимо быть внимательными, чтобы избежать переобучения. Подбор оптимальных параметров модели и контроль за глубиной деревьев и числом деревьев в ансамбле могут помочь достичь лучших результатов.

В целом, случайный лес является одним из наиболее популярных и широко применяемых алгоритмов машинного обучения. Его простота в использовании, эффективность и способность решать разнообразные задачи делают его незаменимым инструментом для анализа данных и принятия решений.

Цель данной курсовой работы — реализация алгоритма «Случайный лес» и изучение его применения в задачах линейной регрессии

В ходе данной курсовой работы выполнены следующие задачи:

- изучена документация и техническая литература по исследуемой теме;
- реализован алгоритм регрессионного анализа при использовании метода «Случайного леса»

- разработан программный код приложения с использованием языка программирования R;
- приложение протестировано и результаты собраны;
- изучены правила качественного оформления документации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1. Монтгомери Д.С., Пек Э.А., Вайнинг Г.Г. Введение в анализ линейной регрессии. — Wiley, 2012. — 314 с.
2. Брейман Л. Случайные леса. Машинное обучение. — Springer, 2001. — 273 с.
3. Лиану А., Винер М. Классификация и регрессия с использованием случайного леса — R News, 2002. — 5 с.
4. IBM. What is random forest? [Электронный ресурс]. — Режим доступа: <https://www.ibm.com/topics/random-forest>
5. Кун М., Джонсон К. Прикладное предсказательное моделирование. — Springer, 2013. — 6 с.
6. Хэсти Т., Тибширани Р., Фридман Д. Основы статистического обучения: Добыча данных, вывод и прогнозирование — Springer, 2013. — 40 с.
7. Чаттерджи А., Хади А.С. Регрессионный анализ на примерах — Wiley, 2015. — 162 с.
8. Джеймс Г., Виттен Д., Хэсти Т., Тибширани Р. Введение в статистическое обучение: с применением в R — Springer, 2013. — 74 с.
9. Пал М. Классификатор случайного леса для классификации с помощью дистанционного зондирования — Международный журнал дистанционного зондирования, 2015. — 6 с.
10. Н.А. Богданова, Ю.С. Зыбина, Е.С. Шпакова Использование сингулярного разложения матриц для сжатия электронно-микроскопических изображений — Национальный исследовательский университет «МИЭТ», 2018. — 5 с.

ПРАКТИЧЕСКАЯ ЧАСТЬ

1. RStudio [Электронный ресурс]. — Режим доступа: <https://www.rstudio.com>
2. Random forest from RStudio [Электронный ресурс]. — Режим доступа: <https://rf.rstudio.com>
3. Kaggle. Dataset of songs in Spotify [Электронный ресурс]. – Режим доступа: <https://www.kaggle.com/datasets/mrmorj/dataset-of-songs-in-spotify>

ПРИЛОЖЕНИЯ

Приложение А — программный код.

Приложение А

```
dataset
dataset <- read.csv("music_genre.csv", header = TRUE)
dataset <- subset(dataset, d[1:50005]!=0)
str(dataset)

dataset <- dataset[ , -c(1,16)]

dataset$music_genre <- as.factor(dataset$music_genre)
table(dataset$music_genre)

sapply(dataset, class)
dataset$key <- as.factor(dataset$key)
dataset$mode <- as.factor(dataset$mode)
sapply(dataset, class)

set.seed(321)
ind <- sample(2, nrow(dataset), replace = TRUE, prob = c(0.7, 0.3))
train <- dataset[ind==1,]
test <- dataset[ind==2,]

library(randomForest)
set.seed(1)
model <- randomForest(
  formula = music_genre ~ .,
  data = train
)

model
```

```
plot(model)
```

```
result <- predict(model, newdata=test)
```

```
result <- data.frame(result)
```

```
class(result)
```

```
str(result)
```

```
compare <- ifelse(result$ result == test$ music_genre , 'yes', 'no')
```

```
compare
```

```
accuracy <- sum(compare == "yes") / length(compare)
```

```
accuracy
```

```
model_tuned <- tuneRF(
```

```
  x=dataset[,-1],
```

```
  y=dataset$music_genre,
```

```
  ntreeTry= 500 ,
```

```
  mtryStart= 4 ,
```

```
  stepFactor= 1.5 ,
```

```
  improve= 0.01 ,
```

```
  trace= FALSE
```

```
)
```

```
result_tuned <- predict(model_tuned, newdata=test)
```

```
result_tuned <- data.frame(result_tuned)
```

```
str(result_tuned)
```

```
compare <- ifelse(result_tuned$ result_tuned == test$ music_genre , 'yes', 'no')
```

```
compare
```

```
accuracy <- sum(compare == "yes") / length(compare)
```

```
accuracy
```